

文件I/O

1、C标准函数与系统函数的区别

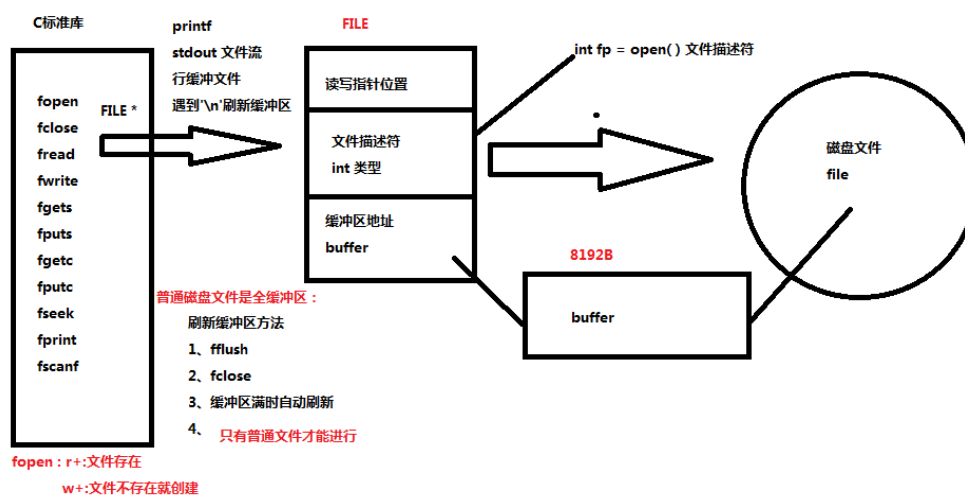
C标准库函数都有一个FILE结构体：读写指针位置，文件描述符，缓冲区地址。

1、缓冲区的刷新方法：（普通磁盘文件是全缓冲文件，必须以下操作才能刷新缓冲区）

1、fflush

2、fclose

3、缓冲区满时，自动刷新。（8192B）



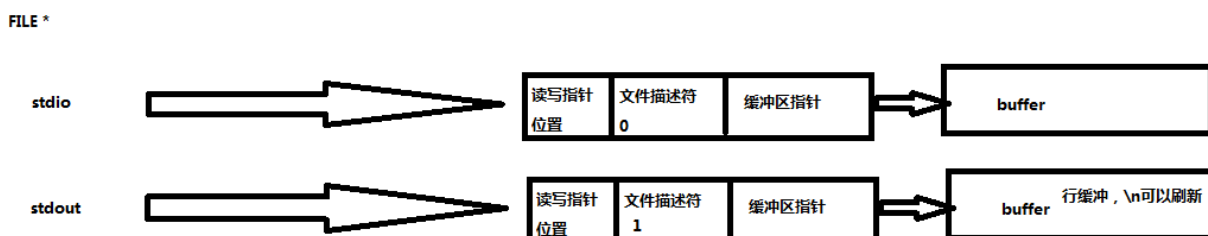
2、行缓冲区：

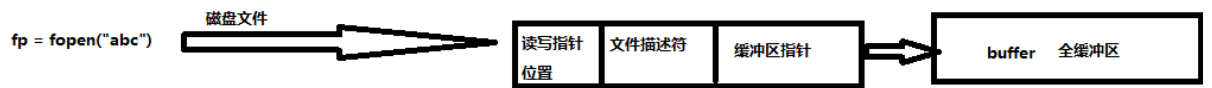
stdin\stdout\stderr等文件流都有一个FILE结构体，是行缓冲文件。（属于库函数的标志）

printf 遇到' \n' 刷新缓冲区，使用' \n' 刷新行缓冲区。

stdin都是一个FILE指针，指向一个FILE结构体：文件描述符 0，键盘输入

stdout都是一个FILE指针，指向一个FILE结构体：文件描述符 1，打印到显示屏





C标准函数通过系统的write等文件操作写到磁盘上。

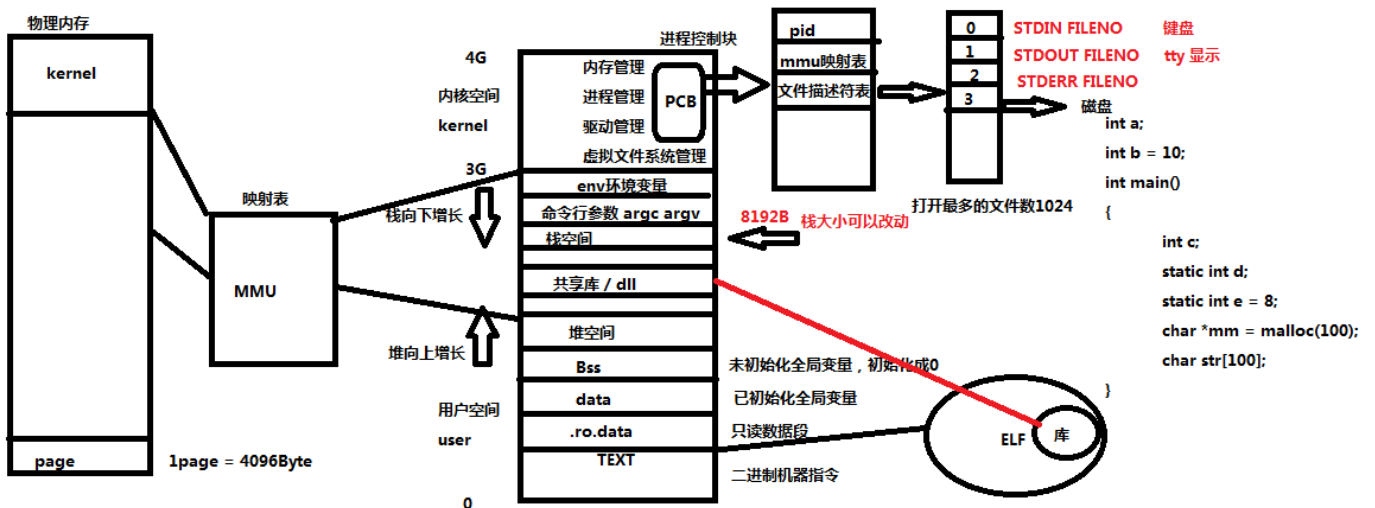
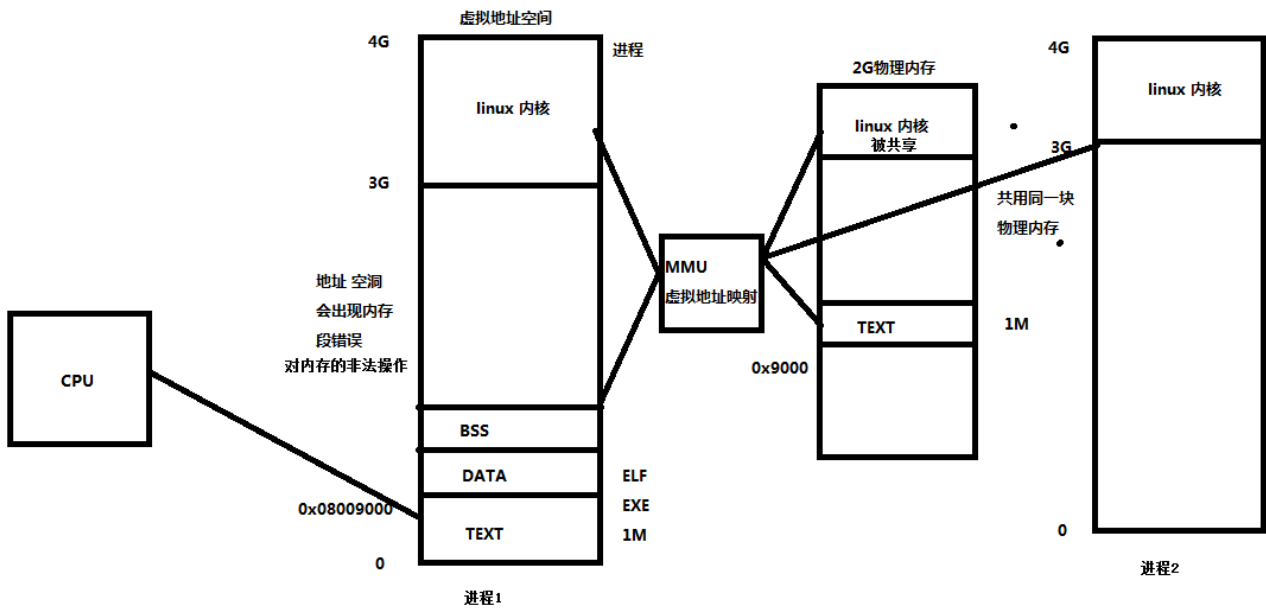
C标准函数打开文件：

R+ 表示打开的文件存在；

W+ 表示打开的文件可以是不存在的。

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     FILE *fp;
6     fp = fopen("abc", "w+");
7     fprintf(fp, "hello\n");
8     fflush(fp);
9     //fclose(stdout);
10
11     while(1)
12         ;
13
14     /*
15     while(1)
16     {
17         printf("hello");
18         usleep(10000);
19     }
20     */
21     return 0;
22
23 }
```

3、应用开发（C标准库）→ Linux 应用层接口函数 → 内核 驱动函数 → 硬件



2、open函数可以三个参数，也可以是两个参数。

当参数有O_CREAT，那么就需要加mode的参数。如果没有mode的话，权限位会出错。

创建文件还需要考虑umask

文件描述符0,1,2已经被占用，每个进程最多能打开1024个文件描述符。

Linux可执行文件是ELF格式。

虚拟地址是由MMU虚拟地址映射而来的，虚拟内存4G。3G~4G给linux内核使用。

每个进程操作系统都会分配0~4G的虚拟内存，linux内核是每个进程在物理内存都是共享，虚拟地址会有空洞，linux内核里都有一个进程控制块PCB。

文件描述符：（属于系统API的标准）

i.STDIN_FILENO 键盘文件等输入

ii.STDOUT_FILENO tty屏幕文件等输出

iii.STDERR_FILENO 屏幕文件等错误输出

O_CREAT 创建文件

O_EXCL 创建文件时，如果文件存在则出错返回

O_TRUNC 把文件截断成0

O_RDONLY 只读

O_WRONLY 只写

O_RDWR 读写

O_APPEND 追加，移动文件读写指针位置到文件末尾

O_NONBLOCK 非阻塞标志

O_SYNC 使每次write 都等到物理I/O操作完成，才进行返回。包括文件属性的更新。

open. c

```
1 #include <stdio.h>
2 #include <sys/types.h>
3 #include <sys/stat.h>
4 #include <fcntl.h>
5 //一次可以打开1024文件描述符
6
7 int main(void)
8 {
9     int fd,i = 3;
10    char name[100];
11    while(1)
12    {
13        sprintf(name,"file%d",i);
14        fd = open(name,O_CREAT,0664);
```

```

15     if(fd < 0)
16         break;
17     printf("%d\n", fd);
18     i++;
19 }
20 return 0;
21 }

```

open_flag.c

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <sys/stat.h>
4  #include <sys/types.h>
5  #include <fcntl.h>
6  #include <unistd.h>
7  #include <errno.h>
8
9  void sys_err(char *str)
10 {
11     perror(str);
12     exit(-1);
13 }
14
15 int main(int argc, char **argv)
16 {
17     int fd;
18     if(argc < 2)
19     {
20         printf("./a.out filename\n");
21         exit(-1);
22     }
23     printf("begin errno:%d\n", errno);
24     fd = open(argv[1], O_RDWR);
25     if(fd < 0)
26     {
27         printf("after errno:%d\n", errno);
28         perror("open");//打印出错的信息
29         exit(-1);
30     }
31     printf("suceess\n");

```

```

32 #if 0
33     fd = open(argv[1],O_TRUNC);
34 #endif
35
36 #if 0
37     fd = open(argv[1],O_CREAT | O_EXCL,0664);
38     if(fd < 0)
39     {
40         printf("open fail\n");
41         exit(-1);
42     }
43     printf("do something\n");
44 #endif
45     close(fd);
46     return 0;
47 }

```

创建文件时，注意权限位。

PCB 进程控制块

每个进程在PCB(Process Control Block)既进程控制块中都保存着一份文件描述符表，文件描述符就是这个表的索引，文件描述表中每个表项都有一个指向已打开文件的指针。文件描述符表中的指针指向file结构体。

file 结构体中维护中 File Status Flag 标记 和当前读写位置，及引用计数。

dup、fork等系统调用会导致多个文件描述符指向同一个file结构体。例如有 fd1 和 fd2 都引用同一个file结构体，那么它的引用计数就是 2

当close (fd1)时并不会释放file结构体，而只是把引用计数减到 1，如果在close (fd2)，引用计数就会减到 0 同时释放file 结构体，这才真的关闭了文件。

task_struct结构体路径：/usr/src/linux-headers-3.13.0-24/include/linux/sched.h

进入 Man Page 选中函数名 然后 系统 为 2，按 2 再 按 K，进入 Man Page

查看宏定义：【+ D

3、查看环境变量env

4、Ulimit -a 配置

```
xingwenpeng@ubuntu:~/LinuxCode/IOBase-01$ ulimit -a
core file size          (blocks, -c) 0
data seg size           (kbytes, -d) unlimited
scheduling priority     (-e) 0
file size               (blocks, -f) unlimited
pending signals         (-i) 7726
max locked memory       (kbytes, -l) 64
max memory size         (kbytes, -m) unlimited
open files              (-n) 1024
pipe size               (512 bytes, -p) 8
POSIX message queues    (bytes, -q) 819200
real-time priority      (-r) 0
stack size              (kbytes, -s) 8192
cpu time                (seconds, -t) unlimited
max user processes      (-u) 7726
virtual memory          (kbytes, -v) unlimited
file locks              (-x) unlimited
```

5、一个内存页 4096 Byte

6、int close(int fd);函数：成功返回零，失败返回-1。

7、<error.h>需要查看error时，就需要导入头文件。

8、新打开文件返回文件描述符，会使用0,1,2，之外未使用的最小文件描述符。

9、read函数从打开的设备或文件中读取数据

```
1 #include<unistd.h>
2
3 ssize_t read(int fd,void *buf,size_t count);
```

返回值：成功返回读取的字节数，出错返回-1,并设置error,如果在调read之前到达文件末尾，则这次read返回0.

参数count是请求读取的字节数，读上来的数据保存在缓冲区buf中，同时文件的当前读写位置向后移。

- 1、 请求返回多少个字节数，就返回多少个字节数；
- 2、 返回的字节个数小于请求的返回的字节个数：

3、返回零;

4、返回-1。

10、write函数向打开的设备或文件中写数据。

```
1 ssize_t write(int fd,const void *buf,size_t count);
```

返回值：成功返回写入的字节数，出错返回-1并设置error

文件复制代码：

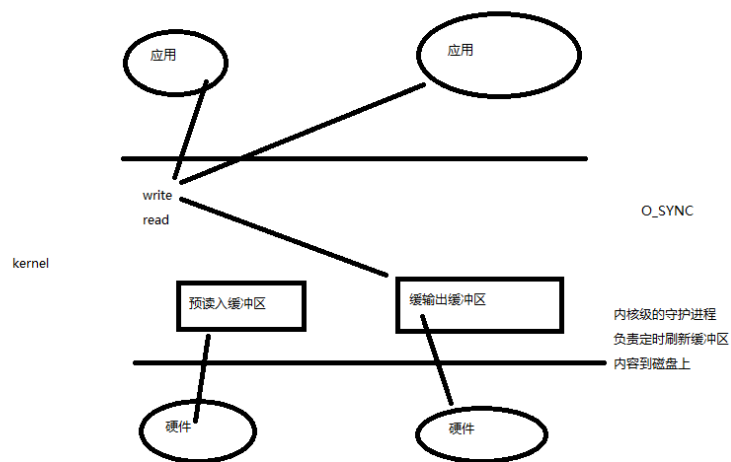
```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/types.h>
5 #include <sys/stat.h>
6 #include <fcntl.h>
7
8 void sys_err(char *str)
9 {
10     perror(str);
11     exit(1);
12 }
13
14 int main(int argc,char *argv[])
15 {
16     int srcfd,destfd,len;
17     char buf[4096];
18     if(argc < 3)
19     {
20         printf("./mycp srcname destname\n");
21         exit(1);
22     }
23     srcfd = open(argv[1],O_RDONLY);//打开源文件
24     if(srcfd < 0)
25     {
26         sys_err("open src");
```

```

27     }
28     destfd = open(argv[2],O_CREAT|O_WRONLY|O_TRUNC,0664);//目标文件
29     if(destfd < 0)
30     {
31         sys_err("open dest");
32     }
33
34     while((len = read(srcfd,buf,sizeof(buf))) > 0)//开始读
35     {
36         write(destfd,buf,len);//写文件
37     }
38     close(srcfd);
39     close(destfd);
40
41     return 0;
42 }

```

读写操作在内核里有预读\预写入缓冲区，为了减少多次读写磁盘。内核设有守护进程，负责定时刷新缓冲区，内容到磁盘上。



O_SYNC使每次write 都等到物理I/O操作完成，才进行返回。

时间太长，但安全。

11、阻塞和非阻塞

1、读阻塞

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <sys/types.h>
4 #include <sys/stat.h>
5 #include <fcntl.h>
6 #include <unistd.h>
7
8 void sys_err(char *str)
9 {
10     perror(str);
11     exit(1);
12 }
13
14 #if 0
15
16 int main(int argc, char *argv[])
17 {
18     int fd, len;
19     char buf[4096];
20     while(1)
21     {
22         len = read(STDIN_FILENO, buf, sizeof(buf)); //读阻塞
23         printf("hello\n");
24         write(STDOUT_FILENO, buf, len);
25     }
26
27     return 0;
28 }
29
30 #endif
31
32
33 int main(int argc, char *argv[])
34 {
35
36     int fd, len;
37     char buf[5];
38     len = read(STDIN_FILENO, buf, sizeof(buf));
39     write(STDOUT_FILENO, buf, len);
40     return 0;
```

```
41 }
```

2、非阻塞：O_NONBLOCK 非阻塞，是文件的属性，并非是read和write的阻塞。

C标准库读入数据带'\0' ;而UNIX标准函数读入（read）不带'\0' .

/dev/tty 表示当前终端的指针

```
1 #include <unistd.h>
2 #include <fcntl.h>
3 #include <errno.h>
4 #include <string.h>
5 #include <stdlib.h>
6
7 #define MSG_TRY "try again\n"
8 #define MSG_TIMEOUT "timeout\n"
9
10 void sys_err(char *str)
11 {
12     perror(str);
13     exit(-1);
14 }
15
16 int main(void)
17 {
18     char buf[10];
19     int fd,n,i;
20     fd = open("/dev/tty",O_RDONLY|O_NONBLOCK);
21     if(fd < 0)
22     {
23         sys_err("open /dev/tty");
24     }
25     for(i = 0; i < 5; i++)
26     {
27         n = read(fd,buf,10);
28         if(n >= 0)
29             break;
30         if(errno != EAGAIN)
31         {
32             sys_err("read /dev/tty");
```

```

32         sys_err( "read /dev/tty" );
33     }
34     sleep(1);
35     write(STDOUT_FILENO,MSG_TRY,strlen(MSG_TRY));
36 }
37 if(i == 5)
38     write(STDOUT_FILENO,MSG_TIMEOUT,strlen(MSG_TIMEOUT));
39 else
40     write(STDOUT_FILENO,buf,10);
41
42 close(fd);
43 return 0;
44 }

```

非阻塞模式：

```

1  #include <stdio.h>
2  #include <unistd.h>
3  #include <sys/stat.h>
4  #include <sys/types.h>
5  #include <stdlib.h>
6  #include <fcntl.h>
7
8  void sys_err(char *str)
9  {
10     perror(str);
11     exit(-1);
12 }
13 int main(int argc,char *argv[])
14 {
15     int len,i = 0;
16     char buf[50];
17     //1、STDIN_FILENO默认是阻塞属性
18     close(STDIN_FILENO);
19     //2、open得到新的文件描述符为0,即STDIN_FILENO,设置了非阻塞属性
20     open("/dev/tty",O_RDONLY|O_NONBLOCK);
21     while(1)
22     {
23         len = read(STDIN_FILENO,buf,sizeof(buf));
24         write(STDOUT_FILENO,buf,len);

```

```

25     sleep(1);
26     printf("%d\n",i++);
27 }
28
29 return 0;
30 }

```

read函数读取的字符串，不带 '\0' .

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <sys/types.h>
4 #include <sys/stat.h>
5 #include <fcntl.h>
6 #include <unistd.h>
7
8 void sys_err(char *str)
9 {
10     perror(str);
11     //exit(1);
12 }
13
14 int main(int argc,char *argv[])
15 {
16     int fd,len,i = 0;
17     char buf[1024];
18
19     //STDIN_FILENO默认是阻塞的属性
20     close(STDIN_FILENO);
21
22     //open得到新的文件描述符为零，即STDIN_FILENO,设置了非阻塞属性
23     open("/dev/tty",O_RDONLY|O_NONBLOCK);
24
25     while(1)
26     {
27         len = read(STDIN_FILENO,buf,sizeof(buf));
28         if(len > 0)
29         {
30             if(strncmp("yes",buf,3) == 0)
31             {
32                 printf("ok");

```

```

32         printf("OK\n");
33         break;
34     }
35 }
36
37 write(STDOUT_FILENO,buf,len);
38 sleep(1);
39 printf("%d\n",i++);
40 }
41 return 0;
42 }

```

12、lseek函数

```
1 off_t lseek(int fd,off_t offset,int whence);
```

lseek成功时返回当前偏移量，失败的时候返回-1，并将**errno**设置为**ESPIPE**.

fseek成功时返回0，失败时返回-1.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/types.h>
5 #include <sys/stat.h>
6 #include <fcntl.h>
7
8 int main(int argc,char *argv[])
9 {
10
11     int fd,len;
12     char buf[1024];
13     if(argc < 2)
14     {
15         printf("./a.out filename\n");
16         exit(-1);
17     }
18
19     fd = open(argv[1], O_CREAT | O_RDWR, 0664);

```

```

19 fd = open(argv[1],O_CREAT | O_RDWR,0664),
20 write(fd,"hello",5);
21
22 lseek(fd,0,SEEK_SET);
23
24 read(fd,buf,sizeof(buf));//读取没有'\0'
25 printf("%s\n",buf);//可能出现乱码
26 //write(STDOUT_FILENO,buf,5);
27
28 #if 0
29 fd = open(argv[1],O_RDONLY);
30 len = lseek(fd,0,SEEK_END);
31 printf("len = %d\n",len);
32 #endif
33
34 close(fd);
35 return 0;
36
37 }

```

lseek函数可以创建一个指定大小的文件

lseek函数拓展，要在下一次写才能生效。**od -tx filename** 查看文件内容

od命令

用户通常使用od命令查看特殊格式的文件内容。通过指定该命令的不同选项可以以十进制、八进制、十六进制和ASCII码来显示文件。

语法：od [选项] 文件…

命令中各选项的含义：

-A 指定地址基数，包括：

- d 十进制
- o 八进制（系统默认值）
- x 十六进制
- n 不打印位移值

-t 指定数据的显示格式，主要的参数有：

- c ASCII字符或反斜杠序列
- d 有符号十进制数
- f 浮点数
- o 八进制（系统默认值为02）

u 无符号十进制数

x 十六进制数

除了选项c以外的其他选项后面都可以跟一个十进制数n，指定每个显示值所包含的字节数。

1、通过 lseek 获取文件大小，读取内容到字符串

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <stdlib.h>
6 #include <sys/types.h>
7 #include <sys/stat.h>
8 #include <fcntl.h>
9
10 int main(int argc, char **argv)
11 {
12     int fd;
13     fd = open(argv[1], O_RDWR);
14     printf("fd = %d\n", fd);
15     if (fd < 0)
16     {
17         exit(0);
18     }
19
20     int len = lseek(fd, 0, SEEK_END);
21     lseek(fd, 0, SEEK_SET);
22
23     char buf[10] = {0};
24     char *content = (char *)calloc(sizeof(char), len+1);
25     int count = 0;
26     while((len = read(fd, buf, sizeof(buf))) > 0){
27         memcpy(content+count, buf, len);
28         memset(buf, 0, sizeof(buf));
29         count += len;
30     }
31     printf("content = %s\n", content);
32
33     return 0;
34 }
```

2、拓展文件长度

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/stat.h>
5 #include <sys/types.h>
6 #include <fcntl.h>
7
8 void sys_err(char *str)
9 {
10     perror(str);
11     exit(-1);
12 }
13
14 int main(int argc, char *argv[])
15 {
16     if(argc < 3)
17     {
18         printf("./a.out filename size\n");
19         exit(1);
20     }
21
22     int fd = open(argv[1], O_CREAT | O_EXCL | O_RDWR, 0644);
23     int len = atoi(argv[2]);
24
25     if(fd < 0)
26     {
27         sys_err("open");
28     }
29
30     lseek(fd, len-1, SEEK_SET); //下一次写操作，才会真正拓展文件长度
31
32     write(fd, "\0", 1);
33     close(fd);
34     return 0;
35
36 }
```

13、fcntl函数：可以改变一个打开的文件属性，重新设置读、写、追加、非阻塞等标志，而不必重新open文件。

EAGAIN: 当一个文件非阻塞，没有读到数据，返回的错误。

```
1 #include <unistd.h>
2 #include <fcntl.h>
3
4 int fcntl(int fd, int cmd);
5
6 int fcntl(int fd, int cmd, long arg);
7
8 int fcntl(int fd, int cmd, struct flock *lock);
```

获取和设置文件的访问控制属性

F_GETFL 获得文件的属性

F_SETFL 设置文件的属性

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/types.h>
5 #include <sys/stat.h>
6 #include <fcntl.h>
7
8 #if 0
9 int main(void)
10 {
11     int status;
12     int fd = open("abc", O_CREAT | O_WRONLY, 0754);
13     status = fcntl(fd, F_GETFL);
14     if (fd < 0)
15     {
16         perror("open");
17     }
18
19     printf("%d\n", status);
20     if (status & O_RDONLY)
```

```

20     if(status & O_RDONLY)
21         printf("O_RDONLY\n");
22     else if(status & O_WRONLY)
23         printf("O_WRONLY\n");
24     else if(status & O_RDWR)
25         printf("O_RDWR\n");
26
27     close(fd);
28     return 0;
29 }
30 #endif
31
32 int main(void)
33 {
34     int status,len;
35     char buf[1024];
36     //fcntl获取和设置文件描述符的访问控制属性
37     status = fcntl(STDIN_FILENO,F_GETFL);//获取
38     status |= O_NONBLOCK;
39     fcntl(STDIN_FILENO,F_SETFL,status);//设置
40
41     while(1)
42     {
43         len = read(STDIN_FILENO,buf,sizeof(buf));
44         write(STDOUT_FILENO,buf,len);
45         printf("hello\n");
46         sleep(1);
47     }
48     return 0;
49 }

```

- 1、 先通过fcntl获取到文件的属性；
- 2、 然后或上需要添加的属， 比如： O_NONBLOCK
- 3、 再次通过fcntl设置到文件。

14、ioctl函数（物理属性）设置自身的物理属性

```

1 int ioctl(int fd,int request,...);//获取和设置文件特有的物理属性

```

波特率：是每秒传输的位数。 bps

功 能: 检查设备类型， 判断文件描述词是否是为终端机。

用 法: `int isatty(int desc);`

返回值：如果参数`desc`所代表的文件描述词为一终端机则返回1， 否则返回0。

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/ioctl.h>
5
6 int main(void)
7 {
8     struct winsize size;
9     if(isatty(STDOUT_FILENO)==0)
10    {
11        exit(1);
12    }
13
14    if(ioctl(STDOUT_FILENO,TIOCGWINSZ,&size)<0)//获取
15    {
16        perror("ioctl TIOCGWINSZ error");
17        exit(1);
18    }
19    printf("%d rows,%d columns\n",size.ws_row,size.ws_col);
20
21    return 0;
22 }
```